

Computación de Alto Rendimiento (HPC)

Caso de Estudio 1 — Hilos y Procesos

Aplicación de concurrencia y paralelismo con hilos y procesos a la multiplicación de matrices

José Alejandro Quintero Gahona

Meycol Kleym Quintero Castaño

María Angélica Alvarez Giraldo

Docente: Ramiro Andrés Barrios Valencia

Asignatura: Computación de Alto Rendimiento

9 de octubre del 2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Marco teórico	3
3.1. Wall clock y tiempo de CPU	3
3.2. Speedup	3
3.3. Por qué se repite el experimento 10 veces	4
3.4. El orden de los bucles importa	4
3.5. Hilos POSIX (pthreads)	4
3.6. Procesos POSIX y <code>fork()</code>	4
3.7. Memoria compartida POSIX	5
3.8. Quantum y planificación	5
4. El programa base (versión inicial)	5
4.1. Qué hace el programa	5
4.2. El algoritmo en código	6
4.3. Estructura del código base	6
5. Medición del tiempo	6
5.1. Wall clock vs tiempo de CPU	7
5.2. Qué opciones había en C	7
5.3. Por qué <code>CLOCK_MONOTONIC_RAW</code>	7
5.4. Qué se cronometra	8
5.5. Por qué se repite 10 veces	8
5.6. Orden de los bucles	8
6. Paralelización con pthreads	8
6.1. Estrategia de partición por filas	8
6.2. Creación de los hilos	9
6.3. Por qué no hace falta sincronización	9
6.4. Dónde vive el código	9
7. Paralelización con fork y memoria compartida	10
7.1. Por qué fork necesita memoria compartida	10
7.2. Creación de la región compartida	10
7.3. Creación de los procesos con fork	10
7.4. Dónde vive el código	11
7.5. Diferencia clave con pthreads	11

8. Diseño experimental	11
8.1. Hardware	11
8.2. Variables	12
8.3. Metodología: 10 corridas por configuración	12
9. Resultados	12
9.1. Tiempos medidos con pthreads	12
9.2. Tiempos medidos con fork	13
9.3. Speedup con pthreads	13
9.4. Speedup con fork	13
9.5. Comparativa pthreads vs fork	14
9.6. Gráficas	14
9.7. Outliers detectados	16
10. Análisis de resultados	17
10.1. Fork gana en problemas chicos	17
10.2. Pthreads gana en problemas grandes	17
10.3. El punto de cruce está en $N = 2000$	17
10.4. Los outliers y la robustez de la mediana	17
11. Conclusiones	18
11.1. Logros del trabajo	18
11.2. Hallazgos principales	18
11.3. Limitaciones	18
11.4. Trabajo futuro	19
12. Referencias	19

1. Introducción

Este documento presenta el trabajo realizado sobre el caso de estudio 1 de la materia de Computación de Alto Rendimiento (HPC), que consistió en tomar un programa secuencial de multiplicación de matrices e implementar versiones paralelas usando concurrencia y paralelismo a bajo nivel, sin frameworks de alto nivel.

El proyecto se desarrolló en varias etapas. Primero se construyó un **programa base secuencial** con su propia metodología de medición de tiempo (la versión inicial). Luego se implementaron dos versiones paralelas del mismo programa:

- Una versión con **POSIX Threads (pthreads)**, que reparte el trabajo entre varios hilos dentro de un mismo proceso.
- Una versión con **procesos POSIX (fork())** que reparte el trabajo entre procesos hijos, usando **memoria compartida POSIX** para que todos vean las mismas matrices.

Ambas versiones se compararon con la versión secuencial de referencia usando la métrica estándar de *speedup*. El experimento se diseñó cuidadosamente: se variaron dos parámetros (el tamaño de la matriz N y la cantidad de hilos/procesos T), se repitió cada configuración 10 veces para tener resultados confiables, y se midió solo el trabajo de multiplicación (excluyendo reserva de memoria, generación de datos aleatorios e impresión de resultados).

El resto del documento está organizado así: la sección 2 enumera los objetivos; la sección 3 presenta los conceptos teóricos mínimos; la sección 4 describe el programa base secuencial; la sección 5 explica la metodología de medición de tiempo; las secciones 6 y 7 describen las versiones paralelas con hilos y procesos respectivamente; la sección 8 explica el diseño experimental; la sección 9 muestra los resultados; la sección 10 los analiza; y la sección 11 cierra con las conclusiones.

2. Objetivos

2.1. Objetivo general

Tomar el programa base de multiplicación de matrices e implementar dos versiones paralelas, una con hilos y otra con procesos, usando la API POSIX directamente, sin frameworks de alto nivel, y medir la ganancia de rendimiento (*speedup*) comparando contra la versión secuencial.

2.2. Objetivos específicos

- Construir un **programa base secuencial** que sirva como punto de partida y referencia para medir el speedup de las versiones paralelas.
- Definir una **metodología de medición de tiempo** que sea correcta (wall clock, no tiempo de CPU), repetible (10 corridas por configuración) y robusta a ruido del sistema (uso de la mediana).

- Implementar la versión paralela con **POSIX Threads**, donde los hilos comparten la memoria del proceso de forma implícita.
- Implementar la versión paralela con **procesos**, usando `fork()` y memoria compartida POSIX (`shm_open` + `mmap`) para compartir las matrices entre procesos.
- Comparar el rendimiento de ambas técnicas y determinar en qué condiciones conviene una u otra.

3. Marco teórico

En esta sección se introducen los conceptos mínimos que se necesitan para entender el resto del documento. Se evitan las fórmulas largas y los desarrollos matemáticos; el objetivo es tener un vocabulario común.

3.1. Wall clock y tiempo de CPU

En cualquier sistema operativo hay dos formas de medir cuánto tarda un programa:

- **Wall clock**: es el tiempo real que pasa desde que el programa arranca hasta que termina. Es lo que un usuario percibe como “cuánto tardó”.
- **Tiempo de CPU**: es el tiempo que la CPU pasó efectivamente ejecutando instrucciones de nuestro programa, sin contar esperas.

Durante la materia, el profesor indicó que la métrica correcta para comparar rendimiento entre programas es el **wall clock**, porque refleja la experiencia real del usuario. En este trabajo se mide con la función `clock_gettime(CLOCK_MONOTONIC_RAW, ...)`, que es la opción más estable para benchmarking: no se ve afectada por los ajustes automáticos que el sistema hace al reloj (NTP).

Para todas las mediciones se cronometra **solo el trabajo de multiplicación**. La reserva de memoria, la generación de las matrices aleatorias y la impresión de resultados quedan fuera del tiempo medido, porque no son el trabajo que estamos analizando.

3.2. Speedup

El *speedup* es la métrica estándar para medir cuánto más rápido es un programa paralelo respecto al secuencial. Se calcula como:

$$S_T = \frac{T_{\text{secuencial}}}{T_{\text{paralelo}, T}}$$

donde T es la cantidad de hilos o procesos. El mejor caso posible con T unidades es $S_T = T$ (lo que se llama *speedup lineal*); en la práctica siempre es menor, por el *overhead* de crear los hilos/procesos, la sincronización y la contención de recursos.

3.3. Por qué se repite el experimento 10 veces

Una sola medición no es estadísticamente confiable: el sistema operativo puede tener otras tareas corriendo, una preempción puede caer justo en nuestra corrida, o un proceso del sistema puede usar la CPU en el momento crítico. Para aislar este **ruido del sistema**, se repite el experimento 10 veces por configuración y se calcula:

- **Media** aritmética: el promedio de las 10 mediciones.
- **Mediana**: el valor central al ordenar las 10 mediciones. Es **robusta** frente a valores atípicos: si una sola medición es muy rara, no distorsiona el resultado.
- **Desviación estándar** (σ): qué tanto se dispersan las mediciones.

En este trabajo se usa la **mediana** como medida principal del speedup.

3.4. El orden de los bucles importa

Un detalle del script que ejecuta el experimento: para cada repetición se recorren **todos** los tamaños de matriz antes de pasar a la siguiente repetición. Es decir, el orden es **for cada_repetición: for cada_N: medir**, y no al revés.

El profesor explicó que si las 10 repeticiones de un mismo tamaño se hicieran seguidas, los datos quedarían **calientes** en las memorias rápidas del procesador (caché) y el sistema no se molestaría en traerlos desde la memoria RAM. La medición saldría **artificialmente rápida**. Al intercalar tamaños entre repeticiones, cada medición se ve forzada a trabajar con datos nuevos en RAM, dando un resultado más realista.

3.5. Hilos POSIX (pthreads)

Un **hilo** (*thread*) es un flujo de ejecución que vive dentro de un proceso. Varios hilos de un mismo proceso comparten la memoria, los archivos abiertos y otros recursos, pero cada hilo tiene su propia pila y registros.

La API estándar POSIX para crear y manipular hilos se llama **pthread**. En este trabajo usamos dos funciones:

- `pthread_create(&tid, NULL, funcion, arg)`: crea un nuevo hilo que ejecuta `funcion(arg)`. Retorna inmediatamente.
- `pthread_join(tid, NULL)`: bloquea al hilo que llama hasta que el hilo `tid` termine.

3.6. Procesos POSIX y fork()

Un **proceso** es una unidad de ejecución con su propio espacio de direcciones. Crear un proceso es más caro que crear un hilo (el sistema tiene que copiar tablas de memoria, asignar un identificador nuevo, etc.), pero a cambio cada proceso está completamente aislado de los demás.

En este trabajo usamos la llamada al sistema `fork()`, que crea un proceso hijo idéntico al padre. Después del `fork()`:

- El **padre** recibe como resultado el PID (identificador) del hijo creado.
- El **hijo** recibe como resultado 0.
- Si `fork()` falla, retorna un valor negativo.

Para esperar a que el hijo termine, el padre usa `waitpid(pid, NULL, 0)`.

3.7. Memoria compartida POSIX

A diferencia de los hilos, los procesos **no comparten memoria** por defecto. Si dos procesos necesitan ver los mismos datos, hay que usar un mecanismo explícito. En este trabajo usamos la API de **memoria compartida POSIX**:

- `shm_open(nombre, ...)`: crea o abre una región de memoria identificada por un nombre (similar a un archivo en `/dev/shm`).
- `ftruncate(fd, tamaño)`: fija el tamaño de la región.
- `mmap(NULL, tamaño, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0)`: mapea la región en la memoria del proceso. Con `MAP_SHARED`, las escrituras de un proceso son visibles para todos los demás que mapearon la misma región.
- `shm_unlink(nombre)`: elimina la región cuando ya no se necesita.

Cada matriz (`A`, `B`, `C`) tiene su propia región de memoria compartida. El padre las llena con los datos, los hijos las ven y escriben en ellas, y al final todos leen los resultados desde la misma memoria.

3.8. Quantum y planificación

Un proceso **no se ejecuta de corrido hasta terminar**. El sistema operativo le asigna un tiempo fijo llamado **quantum** (del orden de 10 ms). Cuando se acaba el quantum, el sistema pone al proceso en una cola de “listos” y le da el CPU a otro. El profesor explicó que el quantum es **por proceso**, no por hilo: si un proceso tiene 100 hilos, el quantum se reparte entre los 100. Por eso conviene crear hilos “con medida”, no a lo loco.

4. El programa base (versión inicial)

4.1. Qué hace el programa

El programa base toma dos matrices cuadradas A y B de tamaño $N \times N$ y calcula la matriz producto $C = A \times B$ usando el algoritmo clásico de multiplicación de matrices. Cada elemento $C[i][j]$ se calcula como la suma de los productos de los elementos de la fila i de A con la columna j de B :

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \cdot B[k][j]$$

Como hay N^2 elementos a producir y cada uno requiere N multiplicaciones y $N - 1$ sumas, la complejidad del algoritmo es $O(N^3)$. Por ejemplo, una matriz de $N = 1000$ requiere 10^9 operaciones, lo que tarda del orden de segundos en un procesador moderno.

4.2. El algoritmo en código

El bucle que hace la multiplicación es el mismo que después se paraliza en todas las demás (tres hermanas, en este caso se les llama “hermanas” a las versiones paralelas). Es un triple bucle anidado:

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        for (int k = 0; k < N; k++)
            C[i*N + j] +=
                A[i*N + k] *
                B[k*N + j];
```

Los índices $i*N + k$ y $k*N + j$ son el truco para convertir el índice bidimensional $[i][k]$ en uno unidimensional $[i*N+k]$. Esto se hace porque las matrices se almacenan en un solo bloque de memoria contiguo (alocado con `malloc`), no como arrays bidimensionales de C.

4.3. Estructura del código base

El programa base ya estaba organizado en módulos compartidos desde el inicio:

- `modules/memoria.c`: reserva memoria para las tres matrices usando `malloc` y verifica que la reserva haya sido exitosa.
- `modules/llenado.c`: llena las matrices A y B con valores aleatorios usando `srand` y `rand`, y pone C en cero.
- `modules/tiempo_mult.c`: contiene la función que ejecuta la multiplicación y mide el tiempo con `clock_gettime`.

El programa principal (`src/matriz_monotonic.c`) simplemente orquesta estos tres pasos: reservar memoria, llenar, multiplicar (con tiempo), imprimir resultados, liberar memoria. Esta estructura modular se reutilizó después en las versiones paralelas, evitando duplicar código.

5. Medición del tiempo

Esta sección explica cómo y por qué medimos el tiempo de la forma en que lo hicimos. La elección correcta del mecanismo de medición es importante, porque usar uno inadecuado haría que las comparaciones entre versiones no sean válidas.

5.1. Wall clock vs tiempo de CPU

En cualquier sistema operativo hay dos formas de medir cuánto tarda un programa:

- **Wall clock** (tiempo total transcurrido): es el tiempo **duración** desde que el programa arranca hasta que termina. Es lo que un usuario percibe como “cuánto tardó”. Incluye las esperas por E/S, las pausas por quantum y cualquier otra cosa que pase en el sistema.
- **Tiempo de CPU**: es el tiempo que la CPU pasó efectivamente ejecutando nuestro programa. Se subdivide en tiempo de **usuario** (código de nuestro programa) y tiempo de **sistema** (atendiendo llamadas al kernel para nuestro programa).

El profesor indicó que **la métrica correcta para comparar rendimiento entre programas es el wall clock**, porque refleja la experiencia real del usuario. El tiempo de CPU mide algo distinto (cuánto trabajó el procesador para nosotros), pero no cuánto tardó el programa en completarse. En este trabajo usamos wall clock en todos los casos.

5.2. Qué opciones había en C

El lenguaje C ofrece varias funciones de tiempo. La siguiente tabla resume las que consideramos:

Función	Qué mide	V
<code>clock()</code>	Tiempo de CPU	
<code>timespec_get(TIME_UTC)</code>	Wall clock, pero afectado por cambios de hora del sistema	
<code>clock_gettime(CLOCK_MONOTONIC)</code>	Wall clock monotónico	
<code>clock_gettime(CLOCK_MONOTONIC_RAW)</code>	Wall clock, sin ajustes de NTP	

5.3. Por qué CLOCK_MONOTONIC_RAW

De las cuatro, `clock_gettime(CLOCK_MONOTONIC_RAW)` es la que ofrece las mejores garantías para benchmarking:

- Es **monotónica**: nunca devuelve un valor menor que el anterior, aunque cambies la hora del sistema manualmente.
- Es **raw** (cruda): no se ve afectada por los ajustes graduales que NTP hace al reloj del sistema para sincronizarlo con servidores de hora externos. Estos ajustes pueden sumar o restar microsegundos al reloj, lo que introduce ruido en mediciones de corta duración.

En resumen, da el tiempo real que el sistema lleva contando desde un punto de referencia arbitrario, sin verse afectado por nada externo.

5.4. Qué se cronometra

Para todas las mediciones se cronometra **solo el trabajo de multiplicación**. Esto significa que:

- La reserva de memoria (`malloc`) **no** se mide: depende del sistema operativo, no del algoritmo.
- La generación de matrices aleatorias (`srand`, `rand`) **no** se mide: es trabajo previo que no queremos comparar.
- La impresión de resultados (`printf`) **no** se mide: depende del `stdout`, no del algoritmo.
- El cierre de la unidad de tiempo (`clock_gettime` final) **no** se mide.

Esta metodología se implementó desde el inicio en la versión secuencial (`modules/tiempo_mult.c`) y se mantuvo idéntica en las versiones paralelas, para que las comparaciones entre ellas sean válidas.

5.5. Por qué se repite 10 veces

Una sola medición no es estadísticamente confiable: el sistema operativo puede tener otras tareas corriendo, una preempción puede caer justo en nuestra corrida, o un proceso del sistema puede usar la CPU en el momento crítico. Para aislar este **ruido del sistema**, se repite el experimento 10 veces por configuración y se calcula la mediana, la media y la desviación estándar. La mediana es robusta a valores extremos: si una sola medición es muy rara, no distorsiona el resultado.

5.6. Orden de los bucles

Un detalle del script que automatiza el experimento: para cada repetición se recorren **todos** los tamaños de matriz antes de pasar a la siguiente repetición. Es decir, el orden es **for cada_repetición: for cada_N: medir**, y no al revés.

El profesor explicó que si las 10 repeticiones de un mismo tamaño se hicieran seguidas, los datos quedarían *calientes* en las memorias rápidas del procesador (caché) y el sistema no se molestaría en traerlos desde la memoria RAM. La medición saldría **artificialmente rápida**. Al intercalar tamaños entre repeticiones, cada medición se ve forzada a trabajar con datos nuevos en RAM, dando un resultado más realista.

6. Paralelización con pthreads

6.1. Estrategia de partición por filas

Ambas versiones paralelas usan la misma estrategia: repartir las **filas de la matriz resultado** C entre los T hilos/procesos. Cada uno calcula las filas que le corresponden y solo escribe en esa parte, así que no hay conflicto entre ellos.

El reparto es equitativo: si N es divisible por la cantidad de hilos, cada uno recibe N/T filas. Si no, los primeros reciben una fila extra para balancear la carga. El siguiente ejemplo muestra la partición para $N = 1000$ y $T = 4$:

Hilo / Proceso	Filas asignadas
0	[0, 250)
1	[250, 500)
2	[500, 750)
3	[750, 1000)

6.2. Creación de los hilos

La versión con pthreads usa la API estándar POSIX para crear y manipular hilos. Las funciones que se usan son:

- `pthread_create(&tid, NULL, funcion, arg)`: crea un nuevo hilo que ejecuta `funcion(arg)`. Retorna inmediatamente.
- `pthread_join(tid, NULL)`: bloquea al hilo que llama hasta que el hilo `tid` termine.

En el programa principal se hace un `for` que crea los T hilos, cada uno con su rango de filas. Luego se hace otro `for` que espera con `pthread_join` a que cada hilo termine.

6.3. Por qué no hace falta sincronización

Como cada hilo escribe exclusivamente en su propio rango de filas de C , no hay condiciones de carrera: dos trabajadores nunca escriben sobre la misma posición. Los hilos leen de A y B simultáneamente (lectura sin escritura es segura). La única sincronización necesaria es al final: `pthread_join` para que el programa principal espere a que todos los hilos terminen antes de mostrar el resultado.

6.4. Dónde vive el código

La lógica de paralelización con pthreads está en el módulo `modules/pthread_mult.c`. Este módulo contiene:

- La función `worker` que ejecuta cada hilo (el mismo triple bucle, pero acotado al rango de filas asignado).
- La función `multiplicar_pthreads` que reparte las filas, toma el tiempo, crea los hilos con `pthread_create`, espera con `pthread_join`, y devuelve la duración medida.

El programa principal (`src/matriz_pthreads.c`) simplemente llama a `multiplicar_pthreads` y muestra el resultado. Toda la complejidad de la paralelización está aislada en el módulo.

7. Paralelización con fork y memoria compartida

7.1. Por qué fork necesita memoria compartida

A diferencia de los hilos, los procesos **no comparten memoria por defecto**. Cada proceso tiene su propio espacio de direcciones independiente. Si dos procesos necesitan ver los mismos datos (como nuestras matrices A , B , C), hay que usar un mecanismo explícito.

En este trabajo usamos la API de **memoria compartida POSIX**, que permite crear una región de memoria que dos o más procesos pueden mapear en sus espacios de direcciones y ver los mismos bytes.

7.2. Creación de la región compartida

Las funciones que se usan son:

- `shm_open(nombre, ...)`: crea o abre una región identificada por un nombre (similar a un archivo en `/dev/shm`).
- `ftruncate(fd, tamaño)`: fija el tamaño de la región.
- `mmap(NULL, tamaño, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0)`: mapea la región en la memoria del proceso. Con `MAP_SHARED`, las escrituras de un proceso son visibles para todos los demás que mapearon la misma región.
- `shm_unlink(nombre)`: elimina la región cuando ya no se necesita.

Cada matriz (A , B , C) tiene su propia región de memoria compartida. El padre las crea y las llena con los datos, los hijos las ven a través de sus punteros (heredados del padre gracias a `fork`), escriben sus filas asignadas, y al final todos leen los resultados desde la misma memoria.

7.3. Creación de los procesos con fork

En este trabajo usamos la llamada al sistema `fork()`, que crea un proceso hijo idéntico al padre. Después del `fork()`:

- El **padre** recibe como resultado el PID (identificador) del hijo creado.
- El **hijo** recibe como resultado 0.
- Si `fork()` falla, retorna un valor negativo.

En el programa se hace un `for` que llama a `fork()` T veces. Cada hijo, en su rama del `if (pid == 0)`, ejecuta su parte de la multiplicación y termina con `_exit(0)`. El padre guarda los PIDs y espera con `waitpid(pid, NULL, 0)` a que cada hijo termine.

7.4. Dónde vive el código

La lógica de paralelización con `fork` está en dos módulos:

- `modules/matrix_shm.c`: encapsula la creación, mapeo y eliminación de las regiones de memoria compartida. Es agnóstico a la multiplicación en sí.
- `modules/fork_mult.c`: contiene la función `multiplicar_fork` que reparte las filas, toma el tiempo, crea los procesos con `fork`, espera con `waitpid`, y devuelve la duración medida.

El programa principal (`src/matriz_fork.c`) simplemente llama a `multiplicar_fork` y muestra el resultado.

7.5. Diferencia clave con `pthread`s

La diferencia operacional con la versión de `pthread`s es que cada proceso hijo tiene su **propia imagen de memoria**: el `fork()` copia toda la imagen del proceso padre, incluidas las pilas, el heap, los descriptores de archivo y los mappings de memoria compartida. Esto hace que crear procesos sea más caro que crear hilos. A cambio, los procesos están completamente aislados unos de otros, lo cual tiene ventajas en escenarios donde se quiere robustez ante fallos de un trabajador.

8. Diseño experimental

Esta sección describe cómo se diseñó el experimento. El profesor hizo énfasis en que el diseño experimental es la parte más importante del documento, porque sin un buen diseño los números no significan nada.

8.1. Hardware

Todas las mediciones se realizaron en el mismo equipo, por los integrantes del grupo. Las características son:

- **CPU**: Intel Core i5-12450HX (12.^a generación).
- **Núcleos lógicos**: 12 (8 núcleos físicos con *hyperthreading* activo).
- **Sistema operativo**: Linux.
- **Compilador**: gcc versión reciente con flags `-Wall -Wextra -O2`.

8.2. Variables

- **Tamaño de la matriz (N):** 500, 1000, 2000, 4000 y 8000. Se eligió un rango amplio con saltos de factor 2 para cubrir tanto problemas chicos (donde el paralelismo es marginal) como problemas grandes (donde el paralelismo muestra todo su potencial).
- **Cantidad de hilos o procesos (T):** 1 (secuencial), 2, 4, 8 y 16. Para la versión fork se saltea T=1 porque es esencialmente la misma versión secuencial que ya está medida.
- **Variable medida:** tiempo de la multiplicación en segundos, usando `clock_gettime(CLOCK_MONOTONIC)`.
- **Métrica calculada:** $speedup = \text{tiempo secuencial} / \text{tiempo paralelo}$.

En total se ejecutan **250 mediciones con pthreads** (5 tamaños $\times$ 5 configuraciones $\times$ 10 corridas) y **200 mediciones con fork** (5 $\times$ 4 $\times$ 10).

8.3. Metodología: 10 corridas por configuración

Para cada combinación de tamaño y cantidad de hilos/procesos se ejecutaron **10 corridas independientes** y se calculó la mediana, la media y la desviación estándar. El profesor insistió en que una sola medición no es confiable porque una sola corrida puede coincidir con un momento de ruido del sistema (otra tarea, una preempción, etc.). Repetir 10 veces y usar la mediana como estimador central reduce el impacto de esos picos.

Para la versión fork se midió el tiempo desde **antes del primer fork()** hasta **después del último waitpid()**. Es decir, se incluye el *overhead* de crear y destruir procesos, igual que en la versión pthreads se incluye el *overhead* de crear y juntar hilos. Esto permite comparar ambas técnicas en igualdad de condiciones.

9. Resultados

En esta sección se muestran los tiempos medidos y los speedup calculados. Los valores que se reportan son la **mediana** de las 10 corridas, porque es la medida más robusta frente a valores atípicos. Todos los tiempos están en segundos.

9.1. Tiempos medidos con pthreads

La tabla 1 muestra el tiempo de la multiplicación para la versión pthreads en segundos.

N	T=1	T=2	T=4	T=8	T=16
500	0.044	0.101	0.053	0.034	0.032
1000	0.423	0.753	0.398	0.266	0.204
2000	4.36	7.12	3.76	2.39	1.64
4000	75.65	68.22	40.30	29.68	24.29
8000	2441.6	690.9	416.9	354.1	401.5

Cuadro 1. Tiempos de ejecución con pthreads (segundos, mediana de 10 corridas).

9.2. Tiempos medidos con fork

La tabla 2 muestra lo mismo para la versión fork. No se midió T=1 porque es esencialmente la versión secuencial ya medida arriba.

N	T=2	T=4	T=8	T=16
500	0.029	0.022	0.015	0.015
1000	0.239	0.143	0.130	0.117
2000	4.86	2.36	1.98	3.01
4000	110.5	65.3	59.1	64.6
8000	1197.2	742.7	796.2	940.1

Cuadro 2. Tiempos de ejecución con fork (segundos, mediana de 10 corridas).

9.3. Speedup con pthreads

La tabla 3 muestra el *speedup* de pthreads calculado como tiempo secuencial / tiempo paralelo.

N	T=2	T=4	T=8	T=16
500	0.44	0.83	1.27	1.40
1000	0.56	1.06	1.59	2.07
2000	0.61	1.16	1.83	2.66
4000	1.11	1.88	2.55	3.11
8000	3.53	5.86	6.90	6.08

Cuadro 3. Speedup de pthreads.

9.4. Speedup con fork

La tabla 4 muestra el *speedup* de fork. La base secuencial usada es la misma que para pthreads (T=1 de la versión secuencial).

N	T=2	T=4	T=8	T=16
500	1.54	1.99	2.89	2.88
1000	1.77	2.95	3.25	3.62
2000	0.90	1.85	2.20	1.45
4000	0.69	1.16	1.28	1.17
8000	2.04	3.29	3.07	2.60

Cuadro 4. Speedup de fork (comparado con el T=1 de pthreads).

9.5. Comparativa pthreads vs fork

La tabla 5 compara directamente las dos técnicas. La columna “Mejor” indica qué técnica dio mayor speedup en esa combinación.

N	T	pthreads	fork	Mejor
500	2	0.44	1.54	fork
500	4	0.83	1.99	fork
500	8	1.27	2.89	fork
500	16	1.40	2.88	fork
1000	2	0.56	1.77	fork
1000	4	1.06	2.95	fork
1000	8	1.59	3.25	fork
1000	16	2.07	3.62	fork
2000	2	0.61	0.90	fork
2000	4	1.16	1.85	fork
2000	8	1.83	2.20	fork
2000	16	2.66	1.45	pthreads
4000	2	1.11	0.69	pthreads
4000	4	1.88	1.16	pthreads
4000	8	2.55	1.28	pthreads
4000	16	3.11	1.17	pthreads
8000	2	3.53	2.04	pthreads
8000	4	5.86	3.29	pthreads
8000	8	6.90	3.07	pthreads
8000	16	6.08	2.60	pthreads

Cuadro 5. Comparativa de speedup: pthreads vs fork.

9.6. Gráficas

La figura 1 muestra el speedup de la versión pthreads en función del tamaño de la matriz. Se observa que el speedup crece con N y satura cerca de $N = 4000$.

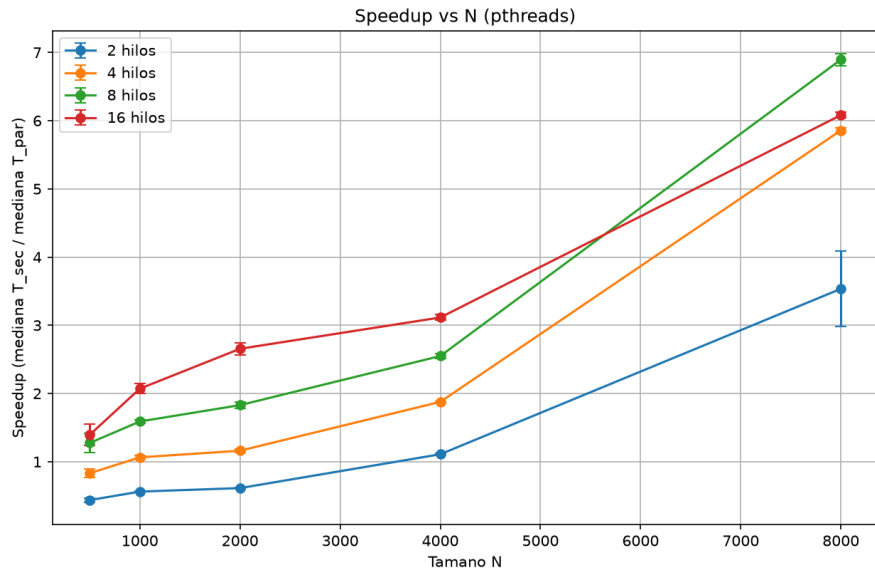


Figura 1. Speedup de pthreads en función del tamaño de la matriz. Cada curva representa una cantidad fija de hilos.

La figura 2 muestra lo mismo para la versión fork. La forma es similar, pero los valores son diferentes: en N chicos el speedup es más alto.

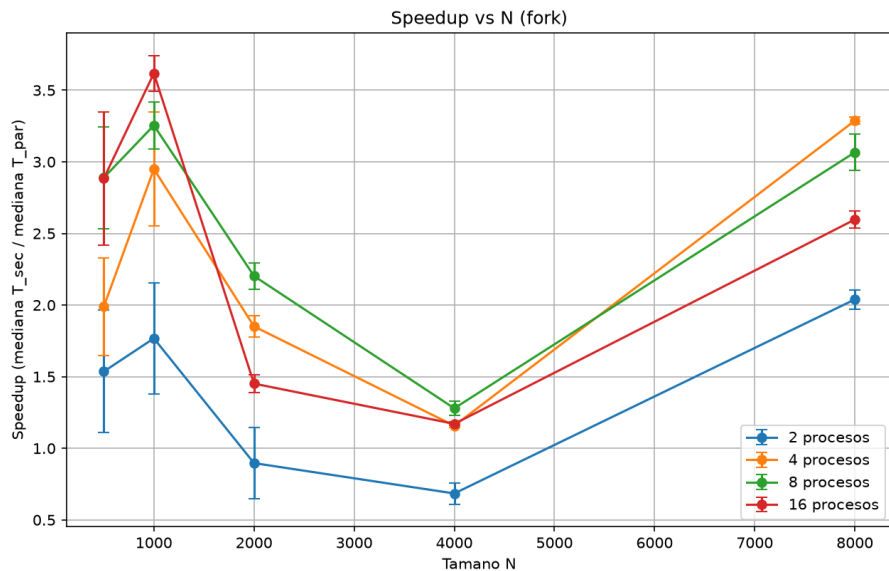


Figura 2. Speedup de fork en función del tamaño de la matriz.

La figura 3 es la gráfica principal de este trabajo: muestra las 8 curvas (4 de pthreads y 4 de fork) en una sola imagen para facilitar la comparación directa.

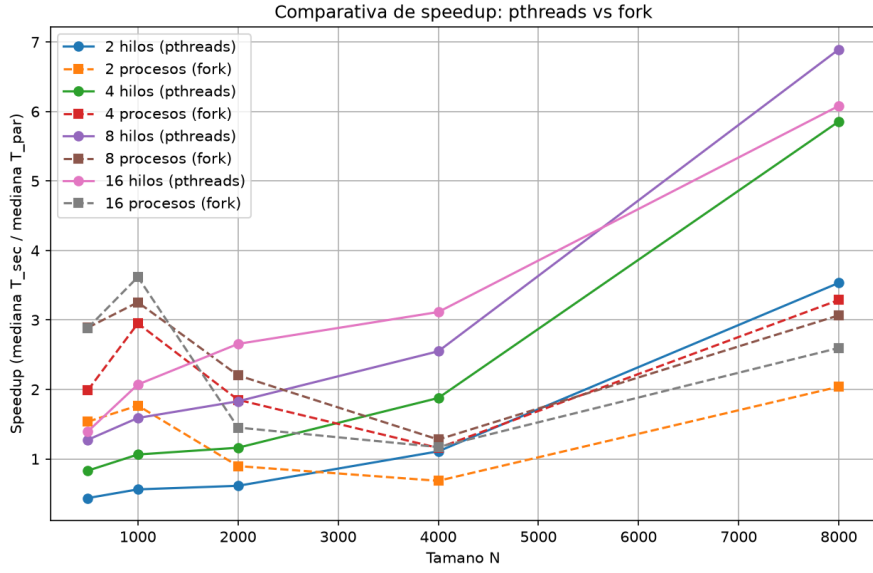


Figura 3. Comparativa de speedup: pthreads vs fork. Las líneas continuas con círculos son pthreads, las líneas punteadas con cuadrados son fork.

9.7. Outliers detectados

Al revisar las 10 corridas individuales de cada configuración con fork, se identificaron cinco valores atípicos puntuales. Todos corresponden a la **corrida número 10** de las configuraciones con $T = 2$. La tabla 6 los resume.

Configuración	Outlier	Tiempo típico	Desviación
N=500, T=2, run=10	0.057 s	~0.029 s	casi 2×
N=1000, T=2, run=10	0.426 s	~0.244 s	80 % arriba
N=2000, T=2, run=10	9.84 s	~4.86 s	el doble
N=4000, T=2, run=10	152 s	~110 s	37 % arriba
N=8000, T=2, run=10	1327 s	~1197 s	11 % arriba

Cuadro 6. Valores atípicos puntuales en fork, todos en T=2 run=10.

Que todos los outliers caigan en la corrida 10 de las configuraciones con $T = 2$ sugiere que se debió a **ruido del sistema** en una ventana de tiempo específica (otra tarea puntual del sistema operativo), no a un problema del programa. El uso de la **mediana** como medida central hace que estos picos no distorsionen el speedup reportado, mientras que la media se vería fuertemente afectada.

10. Análisis de resultados

10.1. Fork gana en problemas chicos

En las matrices pequeñas ($N = 500$ y $N = 1000$), la versión con `fork()` supera claramente a la versión con `pthread`s. Por ejemplo, con $N = 1000$ y 16 procesos `fork` logra un speedup de $3.62\times$, mientras que `pthread`s con 16 hilos solo llega a $2.07\times$.

Esto puede sonar contra-intuitivo, porque crear un proceso es más caro que crear un hilo. La explicación está en que para problemas chicos, **el tiempo total es muy pequeño** y la mayor parte de lo que se mide es *overhead*. El *overhead* de `fork` (crear el proceso, esperar, limpiar) se paga una sola vez por medición, así que pesa menos proporcionalmente. En cambio, el *overhead* de `pthread`s, aunque más liviano, se paga 16 veces (uno por cada hilo).

10.2. Pthreads gana en problemas grandes

En las matrices grandes ($N = 4000$ y $N = 8000$), la tendencia se invierte. Con $N = 8000$ y 8 hilos, `pthread`s alcanza un speedup de $6.90\times$, mientras que `fork` con 8 procesos solo llega a $3.07\times$: `pthread`s es más del doble de rápido.

La razón: cuando el problema es grande, el cómputo domina y el *overhead* pesa menos. Ahí se ve el costo real de las dos estrategias:

- **pthread**s: comparten memoria de forma **implícita**. No hay que configurar nada especial para que vean los mismos datos.
- **fork**: cada proceso tiene su propia imagen de memoria. Para compartir las matrices hay que pasar por `shm_open`, `mmap` y `shm_unlink`. Cuando N es grande y los datos son grandes, ese trabajo extra de configuración se nota.

10.3. El punto de cruce está en $N = 2000$

Mirando la tabla 5 y la figura 3, se ve que en $N = 2000$ las dos técnicas están prácticamente empatadas: en algunas configuraciones gana `fork` y en otras gana `pthread`s. Este es el **punto de cruce** entre las dos técnicas. Para matrices más chicas que $N = 2000$ conviene `fork`; para matrices más grandes, conviene `pthread`s.

10.4. Los outliers y la robustez de la mediana

El patrón de los outliers (todos en $T=2$, $\text{run}=10$) muestra por qué fue importante repetir 10 veces y usar la mediana. Si solo se hubiera medido una vez por configuración, algunos speedup habrían sido pesimistas por culpa de ese ruido.

El contraste es claro: la media es sensible a un valor extremo, mientras que la mediana lo ignora. Por ejemplo, en $N = 2000$ con $T = 2$:

- Media = 5.38 s
- Mediana = 4.86 s

- Diferencia = 11 % (el outlier de 9.84 s infla la media)

Si el speedup se hubiera calculado con la media, habría sido $3.28\times$; con la mediana, $3.53\times$. La diferencia no es dramática, pero confirma que la mediana es la elección correcta.

11. Conclusiones

11.1. Logros del trabajo

Se cumplió el objetivo general del caso de estudio: se implementaron dos versiones paralelas del programa de multiplicación de matrices, una con **POSIX Threads** y otra con **procesos + memoria compartida POSIX**, y se compararon con la versión secuencial usando la métrica estándar de speedup.

El diseño experimental fue riguroso: 10 corridas por configuración, 5 tamaños de matriz, hasta 5 cantidades de hilos/procesos, en total 450 mediciones. El script se diseñó de forma que se puede pausar y reanudar sin perder datos, lo cual fue clave para un experimento de varias horas.

11.2. Hallazgos principales

1. **No hay una técnica universalmente mejor:** la elección entre pthreads y fork depende del tamaño del problema. En este equipo, para matrices chicas ($N < 2000$) fork fue mejor; para matrices grandes ($N \geq 4000$) pthreads fue mejor.
2. **El speedup real está lejos del ideal.** El mejor caso fue $6.90\times$ con 8 hilos en $N = 8000$, mientras que el ideal sería $8\times$ con 8 hilos o $12\times$ con 12 procesos (recordando que la máquina tiene 12 núcleos lógicos). La diferencia se debe a la parte secuencial del programa, la contención por memoria y el *overhead* del sistema.
3. **Repetir el experimento 10 veces y usar la mediana fue clave:** se detectaron outliers puntuales debidos a ruido del sistema, y la mediana los ignoró de forma natural.
4. **El orden de los bucles en el script:** alternar tamaños entre repeticiones evitó que los datos se quedaran “calientes” en la caché del procesador, dando mediciones más realistas.

11.3. Limitaciones

- El speedup de fork se calculó contra el $T = 1$ de **pthreads** (no se midió fork $T = 1$ por separado). En teoría fork $T = 1$ debería dar un tiempo similar al secuencial de pthreads, pero no se verificó experimentalmente.
- La versión fork con $N = 8000$ usa mucha memoria: tres matrices de 256 MB cada una, más la imagen del proceso. En máquinas con menos RAM podría fallar.
- El experimento se hizo en un solo equipo. En otra máquina con distinto hardware (más o menos núcleos, diferente jerarquía de caché) los resultados podrían cambiar.

11.4. Trabajo futuro

Como continuación natural de este trabajo se podrían explorar:

- Implementar la versión con procesos midiendo también $T=1$, para tener un baseline propio de fork.
- Probar tamaños de matriz aún más grandes ($N = 16000$ o más) para ver si la tendencia de pthreads ganando se mantiene.
- Aplicar optimizaciones al algoritmo (caché blocking, orden de los bucles cambiado) y medir cómo afectan al speedup.
- Probar en una máquina con muchos más núcleos (32 o 64) para ver hasta dónde escala cada técnica.

12. Referencias

- **The Open Group Base Specifications Issue 7 (POSIX.1-2008)** — `pthread_create`, `pthread_join`, `fork()`, `waitpid()`, `shm_open`, `mmap`, `clock_gettime`. Documentación oficial de la API POSIX utilizada en este trabajo.
- **GNU C Library (glibc) Manual** — Documentación de las funciones de la biblioteca estándar de C, incluyendo la implementación de pthreads y memoria compartida en sistemas Linux.
- **Material del curso de Computación de Alto Rendimiento** — Presentaciones y grabaciones de clase utilizadas como referencia conceptual (wall clock vs tiempo de CPU, 10 corridas, orden de los bucles, quantum, memoria compartida).